Infrastructure Inventory Management Tool — with Proxmox

Enterprise-grade IT asset inventory platform built with React (Ant Design 5), Node.js/Express, and PostgreSQL. Integrates live VMware vSphere and Proxmox VE discovery, Tenable vulnerability data, Nessus Agent status tracking, and ManageEngine endpoint status — all in one unified dashboard.

Features

Asset Management

- **Four Inventory Types** — Asset Inventory, Ext. Asset Inventory, Beijing Assets, Physical & ESXi Servers
- **25+ Fields per Asset** — VM name, hostname, IP, MAC address (required), OS type/version, department, location, asset tag, server status, patching details, and more
- **iDRAC Section** — Serial Number, OME Status, and iDRAC IP Address shown only when iDRAC Enabled is toggled on
- **Encrypted Credentials** — Asset passwords stored with AES-256-GCM; per-user `can_view_passwords` flag enforced at API level
- **Excel Import / Export** — Smart import with preview and diff (Create / Merge / Errors filter), downloadable template, row-by-row validation
- **Field Customization** — Admins can rename labels, reorder fields, change input types, and add custom extra fields per inventory page
- **Recycle Bin** — Soft-delete with restore capability

VM Discovery

- **VMware vSphere** — Discover VMs from multiple vCenter/ESXi hosts; view dashboards, drift detection, ESXi topology, snapshots, stale VMs, MAC lookup
- **Proxmox VE** — Discover QEMU VMs and LXC containers across nodes; view topology, snapshots, drift, stale VMs
- **Add to Inventory** — One-click push of any discovered VM into Ext. Asset Inventory with pre-filled hostname, IP, MAC address, OS, hosted IP

Software Services

- **ManageEngine Status** — Endpoint deployment tracking and status per asset
- **Nessus Agent Status** — Nessus agent install state per asset with Linux curl install support
- **Tenable Report** — Three-tab report: Matched (in both inventories and Tenable), Not in Tenable, Tenable Only. Stat cards, search, source filter, CSV export; admin import drawer for Tenable Excel files. Covers all 4 inventory sources.

Administration

- **Role-Based Access Control** — Superadmin, Admin, Asset Manager, Viewer; custom roles support; per-page access control
- **User Page Control** — Per-user toggle for password visibility and individual page access
- **Superadmin Account** — Hidden from all non-superadmin users at every API layer; full god-mode access
- **Nav Order** — Drag-and-drop sidebar menu reordering stored in localStorage
- **Audit Logs** — Every create / update / delete / import / export / login / view-password event recorded
- **Backup & Restore** — PostgreSQL dump + CSV export per table; scheduled or manual; restore from upload or history
- **Branding** — Custom logo, app name, colour scheme
- **Dropdown Master** — Centralised dropdown values for OS type, location, server status, patching type, etc.
- **Department Tag Ranges** — Controlled asset tag allocation per department
- **Swagger API Docs** — `/api/docs`

Stack

Layer	Technology
Frontend	React 18 + Vite + Ant Design 5 + React Router 6
Backend	Node.js 20 + Express 4 + JWT + bcrypt
Database	PostgreSQL 18 (pg driver, JSONB)
Files	ExcelJS (Excel parse/generate), multer (uploads)
SSH	node-ssh (remote agent install)

Layer	Technology
Crypto	AES-256-GCM via Node.js `crypto` module
Docs	swagger-ui-express
Process	systemd (production)
Proxy	nginx (production)

Project Layout

```
Inventory_Tool - with Proxmox/
├── backend/
│   ├── server.js                    # Express entry point + static SPA serving
│   └── src/
│       ├── bootstrap/               # Superadmin bootstrap on startup
│       ├── config/                  # DB pool, Swagger spec
│       ├── controllers/             # Route handlers
│       │   ├── assetController.js
│       │   ├── extAssetController.js
│       │   ├── beijingAssetController.js
│       │   ├── physicalEsxiController.js
│       │   ├── tenableController.js   # Tenable report + import
│       │   ├── vmwareController.js    # VMware discovery
│       │   ├── proxmoxController.js   # Proxmox discovery
│       │   ├── softwareStatusController.js
│       │   ├── nessusStatusController.js
│       │   ├── userController.js
│       │   ├── customRolesController.js
│       │   └── ...
│       ├── middleware/
│       │   ├── auth.js              # authenticate / authorize / requirePageAccess
│       │   ├── errorHandler.js
│       │   └── validate.js
│       ├── routes/                  # Express routers (one per resource)
│       ├── services/                # Business logic
│       │   ├── inventoryFieldsService.js   # Dynamic field/group registry
│       │   ├── vmwareService.js            # vSphere API
│       │   ├── proxmoxService.js           # Proxmox API
│       │   ├── vmwareSchedulerService.js
```

```
│       │   ├── proxmoxSchedulerService.js
│       │   ├── backupService.js
│       │   └── ...
│       └── utils/
│           ├── crypto.js              # AES-256-GCM encrypt/decrypt
│           ├── sshVerify.js           # SSH agent install helpers
│           └── winInstall.js
├── frontend/
│   └── src/
│       ├── components/
│       │   ├── Layout/AppLayout.jsx   # Sidebar nav with Software Services group
│       │   └── AddToInventoryModal.jsx  # Push discovered VM to Ext. Inventory
│       └── pages/
│           ├── Assets/                # AssetForm, AssetList, AssetView
│           ├── VMwareDiscovery/       # VM list, dashboard, drift, topology, snapsho
│           ├── ProxmoxDiscovery/      # PVE list, dashboard, drift, topology, snapsh
│           ├── TenableReport/         # 3-tab report, import drawer
│           ├── NessusStatus/
│           ├── SoftwareStatus/
│           └── Admin/                 # Users, Roles, NavOrder, UserPageControl, ...
├── db/
│   ├── schema.sql                     # Core tables
│   ├── vmware_schema.sql              # VMware discovery tables
│   ├── proxmox_schema.sql             # Proxmox discovery tables
│   ├── tenable_schema.sql             # tenable_imports + tenable_assets
│   └── vmware_asset_edits.sql         # VM asset editor audit table
└── README.md
```

Database Setup

Apply schemas in order:

```
psql -U postgres -d inventory_new -f db/schema.sql
psql -U postgres -d inventory_new -f db/vmware_schema.sql
psql -U postgres -d inventory_new -f db/proxmox_schema.sql
psql -U postgres -d inventory_new -f db/tenable_schema.sql
psql -U postgres -d inventory_new -f db/vmware_asset_edits.sql
```

The `mac_address VARCHAR(255)` column must exist on all 4 asset tables (added in a migration — run the schema files if upgrading from an earlier version).

Environment Variables

Copy `backend/.env.example` to `backend/.env` and fill in:

```
# Database
DATABASE_URL=postgresql://postgres:your_password@localhost:5432/inventory_new
DB_PASSWORD=your_password

# Auth
JWT_SECRET=<generate: openssl rand -hex 32>
JWT_EXPIRES_IN=12h

# Encryption (AES-256-GCM) — must be 64 hex chars (32 bytes)
ENCRYPTION_KEY=<generate: openssl rand -hex 32>

# CORS (comma-separated; omit for localhost-only default)
CORS_ORIGIN=http://localhost:4000,http://localhost:5173

# Optional: superadmin bootstrap (creates account on first startup if set)
SUPERADMIN_EMAIL=superadmin@example.com
SUPERADMIN_PASSWORD=Sys@2026!

# Backup
PG_DUMP_PATH=/usr/bin/pg_dump
CSV_BACKUP_DIR=/var/backups/inventory

NODE_ENV=production
PORT=4000
```

> **Never commit** `.env` — it is listed in `.gitignore`. Rotate `JWT_SECRET` and `ENCRYPTION_KEY` periodically.

Local Development

```
# 1. Backend
cd backend
cp .env.example .env   # fill in DB creds
npm install
npm run dev            # http://localhost:4000

# 2. Frontend (new terminal)
cd frontend
npm install
npm run dev            # http://localhost:5173  (Vite proxies /api/* → 4000)
```

Production build (single port)

The backend serves the compiled frontend as static files:

```
cd frontend && npm run build   # outputs to frontend/dist
cd ../backend && node server.js  # serves both API and SPA on :4000
```

Ubuntu Server Setup

1. Prerequisites

```
sudo apt update
sudo apt install -y curl ca-certificates gnupg nginx postgresql

# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

2. PostgreSQL

```
sudo -u postgres psql <<EOF
CREATE USER inventory_user WITH PASSWORD 'strong-password-here';
CREATE DATABASE inventory_new OWNER inventory_user;
GRANT ALL PRIVILEGES ON DATABASE inventory_new TO inventory_user;
EOF
```

```
sudo -u postgres psql -d inventory_new -f /opt/inventory/db/schema.sql
sudo -u postgres psql -d inventory_new -f /opt/inventory/db/vmware_schema.sql
sudo -u postgres psql -d inventory_new -f /opt/inventory/db/proxmox_schema.sql
sudo -u postgres psql -d inventory_new -f /opt/inventory/db/tenable_schema.sql
sudo -u postgres psql -d inventory_new -f /opt/inventory/db/vmware_asset_edits.sql
```

3. Deploy

```
sudo mkdir -p /opt/inventory && sudo chown $USER /opt/inventory
git clone https://github.com/raajsharan/Inventory_Tool-with_Proxmox.git /opt/inventor

cd /opt/inventory/backend && cp .env.example .env && nano .env
npm ci --omit=dev

cd /opt/inventory/frontend && npm ci && npm run build
```

4. systemd

```
sudo cp /opt/inventory/deploy/inventory-backend.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now inventory-backend
```

5. nginx (optional — if not using integrated static serving)

```
sudo cp /opt/inventory/deploy/nginx-inventory.conf /etc/nginx/sites-available/invento
sudo ln -s /etc/nginx/sites-available/inventory /etc/nginx/sites-enabled/inventory
sudo nginx -t && sudo systemctl reload nginx
```

For HTTPS: `sudo certbot --nginx -d your.domain.com`

Roles & Access

Role	Access
Superadmin	Full access to everything; hidden from all other users; cannot be deleted

Role	Access
Admin	All CRUD, user management, dropdowns, roles, audit, backup, field customisation
Asset Manager	Create / edit / delete / import / export assets; cannot manage users or roles
Viewer	Read-only access to assets and dashboard
Custom roles	Admins can define custom roles with page-level permissions

Per-user overrides (set in **User Page Control**):

- `can_view_passwords` — allow/deny revealing encrypted asset passwords
- Individual page access toggles per user (overrides role defaults)

API Highlights

Auth

Method	Path	Purpose
POST	`/api/auth/login`	Login, returns JWT
GET	`/api/auth/me`	Current user profile
POST	`/api/auth/me/change-password`	Change password (rate-limited: 5/15 min)

Asset Inventories (same pattern for `/assets`, `/ext-assets`, `/beijing-assets`, `/physical-esxi-servers`)

Method	Path	Purpose
GET	`/api/assets`	List with search / filter / pagination
POST	`/api/assets`	Create asset
GET	`/api/assets/:id`	Get asset
PUT	`/api/assets/:id`	Update asset
DELETE	`/api/assets/:id`	Soft-delete

Method	Path	Purpose
GET	`/api/assets/:id/password`	Reveal encrypted password (requires `can_view_passwords`)
POST	`/api/assets/import`	Excel import
GET	`/api/assets/export`	Export filtered CSV

VM Discovery

Method	Path	Purpose
GET	`/api/vmware/vms`	List discovered VMware VMs
GET	`/api/vmware/vms/export`	Export VM list as CSV
GET	`/api/proxmox/vms`	List discovered Proxmox VMs / containers
GET	`/api/proxmox/vms/export`	Export Proxmox inventory as CSV

Software Services

Method	Path	Purpose
GET	`/api/software-status`	ManageEngine status per asset
GET	`/api/nessus-status`	Nessus agent status per asset
GET	`/api/tenable/report`	Full Tenable report (matched / not-in / tenable-only)
GET	`/api/tenable/total-ips`	Total Tenable IPs + last import metadata
GET	`/api/tenable/imports`	Import history (admin)
POST	`/api/tenable/import`	Upload Tenable Excel file (admin)
DELETE	`/api/tenable/imports/:id`	Delete import (admin)

Administration

Method	Path	Purpose
GET/PUT	`/api/admin/field-config/:pageKey`	Field customisation per inventory page

Method	Path	Purpose
GET	`/api/users`	User list
GET	`/api/roles`	Role list
GET	`/api/audit`	Audit log
GET	`/api/dropdowns`	All dropdown values
GET/POST	`/api/backup/csv`	Backup / restore CSV per table

Full interactive schema: `/api/docs`

Security

Measure	Detail
Password hashing	bcrypt, 12 rounds
Asset credential encryption	AES-256-GCM (`backend/src/utils/crypto.js`)
JWT	HS256, signed with `JWT_SECRET`; configurable expiry
Password access control	`requirePasswordAccess` middleware DB-checks `can_view_passwords` flag per request — role alone is not sufficient
CORS	Restricted to explicit `CORS_ORIGIN` list; no wildcard
Rate limiting	Login: 20 req/15 min; password change: 5 req/15 min
Password strength	Min 8 chars, must include uppercase, lowercase, and a number
File upload validation	Multer `fileFilter` enforces MIME type + extension (Excel only for Tenable import)
Input validation	express-validator on every route
Helmet	HTTP security headers on all responses
SQL injection	All queries use parameterised `$n` placeholders; CSV column headers validated against `/^[a-zA-Z_][a-zA-Z0-9_]{0,63}$/`

Measure	Detail
Superadmin isolation	Filtered from user lists, role lists, and page-control APIs for all non-superadmin requests
Privilege escalation guard	Explicit check blocks non-superadmin from assigning the `superadmin` role
Error responses	500 errors return a generic message in production; details only in development

---

Upgrading

```
cd /opt/inventory
git pull

# Apply any new DB migrations
psql -U postgres -d inventory_new -f db/vmware_schema.sql
psql -U postgres -d inventory_new -f db/proxmox_schema.sql
psql -U postgres -d inventory_new -f db/tenable_schema.sql

cd backend  && npm ci --omit=dev
cd ../frontend && npm ci && npm run build
sudo systemctl restart inventory-backend
```

License

Internal use.